Distributed-Memory Parallelisation of a Barnes–Hut N-Body Simulation

IT4130E Parallel and Distributed Programming

Chu Xuan Minh Nguyen Binh Anh Doan Phuong Khang Nguyen Tien Phat

IT4130E Parallel and Distributed Programming

June 24, 2026

Outline

The N-body problem

- N masses attract each other by Newtonian gravity; advance their motion through time.
- Force on a body sums contributions of all others:

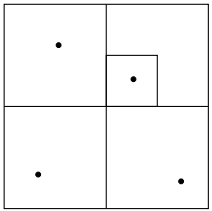
$$\vec{a}_i = G \sum_{j \neq i} m_j \frac{\vec{r}_j - \vec{r}_i}{(\|\vec{r}_j - \vec{r}_i\|^2 + \varepsilon^2)^{3/2}}$$

- Direct evaluation costs $\mathcal{O}(N^2)$ per step — too slow.
- **Goal:** a correct sequential method, a distributed-memory (MPI) parallel version, and a hybrid MPI + OpenMP version, with a measured performance study.

Barnes–Hut: $\mathcal{O}(N \log N)$

- Recursively divide the plane into a **quadtree**; each leaf holds ≤ 1 body.
- Each node stores the total mass and centre of mass below it.
- Force on a body: traverse from the root. If a node is far enough ($s/d < \theta$), treat the whole subtree as one mass; else descend.
- $\theta = 0.5$: about 1% force error, $\mathcal{O}(N \log N)$ work.
- The tree makes the work **irregular** — the key parallel challenge.

$s/d < \theta \Rightarrow$ approximate



Design choices (the rubric questions)

- **Level of parallelism:** *data* — same operation, different bodies.
- **Decomposition:** *data decomposition* (owner-computes per body) over a *recursive* (replicated) tree build $\Rightarrow$ hybrid, data-dominated.
- **Mapping:** *1-D block* — process r owns a contiguous block of $\approx N/P$ bodies (not 2-D $\sqrt{P} \times \sqrt{P}$: the domain is a 1-D list, not a grid).
- **Communication:** *blocking, collective*; broadcast once at start, then one *all-gather* of positions per step. Symmetric single-program multiple-data (SPMD), *no* master–slave; no explicit ring/hypercube topology.
- **Load balance:** equal block *size*; work per body is uneven (dense regions cost more).

Parallel algorithm (process r of P)

1: **(once)** broadcast initial positions, velocities, masses
2: $lo, hi \leftarrow$ block owned by process r ▷ 1-D block
3: **for** each time step **do**
4: build the full quadtree locally ▷ replicated
5: accumulate mass / centre of mass at each node
6: **for** $i = lo \ldots hi - 1$ **do** ▷ OpenMP parallel-for in hybrid
7: $\vec{a}_i \leftarrow$ traverse tree with θ
8: **end for**
9: $\vec{v}_i \mathrel{+}= \vec{a}_i \Delta t; \quad \vec{r}_i \mathrel{+}= \vec{v}_i \Delta t$
10: **all-gather** updated positions
11: **end for**

Force traversal communicates nothing (tree is complete on every process). Cost: build $\mathcal{O}(N)$ replicated + force $\mathcal{O}((N/P)\log N)$ + comm $\mathcal{O}(N)$.

The cluster

- **4 laptops** on a **phone Wi-Fi hotspot**; one Ubuntu VM each (bridged adapter).
- **2 cores per VM** $\Rightarrow$ **8 cores total**, one MPI rank per core.
- Open MPI 4.1.2, Ubuntu 22.04, all nodes equal.
- Round-trip latency $\approx$ 4–5 ms — **this number decides everything that follows.**

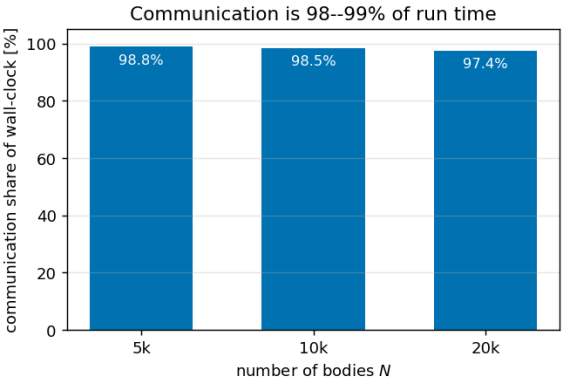
Honest caveat: the hotspot was congested — we report short runs, min-of-repeats, and trust per-step costs rather than absolute wall-clocks.

Node	Cores	MPI
node1	2	4.1.2
node2	2	4.1.2
node3	2	4.1.2
node4	2	4.1.2
total	**8**	

Correctness

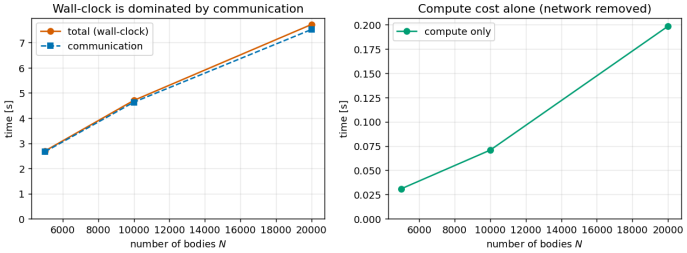
- MPI output matches the sequential reference **bit for bit**:
  - single rank: max difference $= 0$;
  - 8 ranks across all 4 machines: max difference $= 0$ (float tolerance).
- Each body uses identical arithmetic; the all-gather moves data without changing it $\Rightarrow$ the network adds no error.
- Total energy conserved to within **0.6%** over 200 steps $\Rightarrow$ physically stable.

Headline result: communication dominates



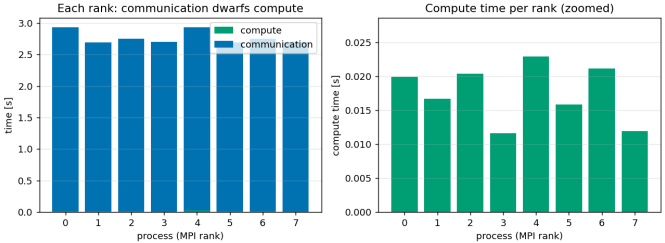
- **2 blocking all-gathers per step** on a 4–5 ms link.
- Communication is **97–99%** of wall-clock at *every* N.
- Latency $\times$ steps, not data volume, is the cost.
- So we always plot **communication and computation separately.**

Running time vs problem size (Experiment A)



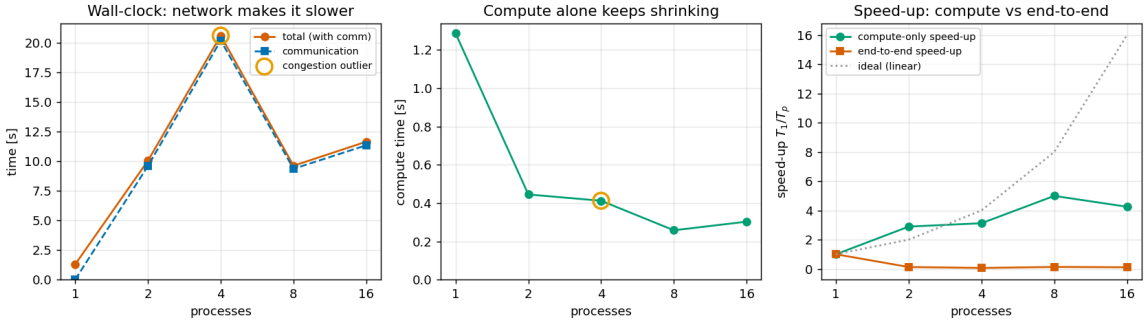
- 8 cores, vary N.
- **Left:** wall-clock $\approx$ communication (they overlap).
- **Right:** compute alone — the true $\mathcal{O}(N \log N)$ cost, $100\times$ smaller.
- Operating size $N = 20{,}000$: $\approx 1\,\text{s/step} \Rightarrow$ 2–3 min $\approx$ 155–185 steps.

Granularity & load balance (Experiment B)



- **Left:** comm (blue) dwarfs compute (green) on every rank.
- **Right:** compute alone — balanced (12–23 ms across ranks).
- Idle-time imbalance $< 25\%$: compute is balanced...
- ...but compute balance barely matters when comm is 99%.

Speed-up at $2N = 40{,}000$ (Experiment C)



Left: end-to-end wall-clock *rises* once the network is used (np=4 is a congestion outlier). **Middle:** compute alone keeps shrinking. **Right:** compute-only speed-up $\approx 5\times$ on 8 cores (near ideal); end-to-end stays < 1 (a slowdown). *The work parallelises; the network does not.*

Conclusion

- Data-parallel Barnes–Hut on a real 4-machine cluster; output **bit-identical** to serial, energy conserved.
- **Communication-bound:** 2 blocking all-gathers/step on a 4–5 ms Wi-Fi link $\Rightarrow$ comm is 97–99% of run time, end-to-end speed-up < 1.
- Separating the costs shows the **computation scales** $\approx 5\times$ **on 8 cores** — the parallelisation is sound; the network is the limit.
- **Fix = communicate less often:** exchange every few steps, overlap with non-blocking comm, or distribute the tree (boundary data only).

Thank you — questions?